

SOFTWARE/ML TASKPHASE REPORT

Prepared by: Rutuj Nilesch Nagarkar

Roll Number: 251090050248

Domain: Software/ML

Repository: https://github.com/RutujN/Synergy_TP_Rutuj

Date: 27 June 2026

Abstract

This report summarizes the Software/ML taskphase completed through five practical tasks. The work began with development-environment setup, Linux commands, dependency management, and version control. It then applied core Python concepts to submission analysis before comparing a manual CSV parser with pandas. A deliberately inconsistent student dataset was cleaned through duplicate removal, type conversion, category standardization, unit normalization, missing-value treatment, and explicit validation. Finally, matplotlib converted the cleaned data into three visual summaries. Together, the tasks demonstrate a reproducible data workflow: source files and dependencies are organized, raw text becomes structured data, quality problems are documented and corrected, and results are communicated through plots. The exercises also show why small functions, clear interfaces, validation checks, and meaningful commits matter: they make analytical work easier to inspect, test, repeat, and extend.

Introduction

The taskphase is designed to connect foundational software practices with a compact data-analysis project. Instead of treating Git, Python, data cleaning, and visualization as isolated topics, the tasks build a single progression. Each output becomes evidence that a concept has been understood and applied. This approach matters in software and machine learning because a model or chart is only as trustworthy as the environment, data, and process that produced it.

Task 1 established the working environment and documented common Linux commands. Task 2 used Python functions and file handling to analyze submission records. Task 3 parsed the same kind of CSV data in two ways: first with basic file operations and then with pandas. Task 4 dealt with problems that appear in real datasets, including duplicated rows, mixed capitalization, number words, missing cells, invalid percentages, and inconsistent units. Task 5 produced plots from the cleaned table. The complete repository therefore records both the code and the reasoning behind a modest end-to-end data pipeline.

Tasks and Main Concepts

Task	Main concept learned
------	----------------------

1	Environment setup, Linux commands, Git, and dependencies
2	Python functions, collections, file handling, and summaries
3	Manual CSV parsing, pandas, aggregation, and comparison
4	Data cleaning, normalization, missing values, and validation
5	Matplotlib charts and evidence-based interpretation

Development Environment & Version Control

Git is a distributed version-control system that records changes as commits. A commit is a named snapshot with an author, timestamp, message, and connection to earlier history. Useful commit messages explain the purpose of a change, allowing another developer to understand how the project evolved. GitHub or another remote host adds collaboration, backup, issue tracking, and review around the Git history.

Branches provide separate lines of development. A feature can be developed and tested without destabilizing the main branch, then merged after review. This is useful even for an individual project because it separates experimentation from accepted work. A `.gitignore` file complements version control by excluding generated output, caches, credentials, and machine-specific files that should not become project history.

A Python virtual environment isolates the interpreter's installed packages for one project. Without isolation, upgrading a library for one assignment can unexpectedly break another. The `requirements.txt` file makes the environment reproducible by declaring dependencies such as pandas, matplotlib, and requests. Another developer can create a fresh environment and install the same project requirements rather than guessing what is missing. Together, Git, a virtual environment, requirements, and ignore rules make the project portable and auditable.

Linux Command-Line Basics

The command line provides a quick and repeatable way to work with project files. Commands such as ``pwd`` and ``ls`` show the current location and its contents, while ``cd`` moves between directories. ``mkdir`` creates folders, and ``touch`` creates an empty file on Linux. ``cat``, ``head``, and ``tail`` inspect text without opening an editor. Searching tools such as ``grep`` locate matching text, and ``find`` locates files by name or other properties.

Linux permissions describe who may read, write, or execute a file. ``chmod`` changes these permissions, but it should be used carefully rather than granting broad access by default. Commands can be combined into repeatable setup or inspection steps, which reduces manual errors. In this taskphase, command-line navigation and file inspection support the Python workflow: the repository root remains the known starting point, input paths are predictable, and generated outputs can be checked immediately after a program runs.

Python & File Handling

Functions divide a program into named units with clear inputs and outputs. In this project, reading data, converting types, calculating summaries, validating fields, saving files, and plotting are separate responsibilities. This separation reduces repetition and makes failures easier to locate. Type hints document expected values—for example, that a path is a string or a summary is a dictionary—while remaining compatible with Python's dynamic execution.

Lists preserve ordered collections and are useful for rows, names, or scores. Dictionaries associate keys with values, making them natural representations of CSV rows and summary metrics. Loops and comprehensions transform these structures, while conditional statements implement rules such as treating ``yes`` as submitted or rejecting attendance outside zero to one hundred.

File input/output connects the program to persistent data. A context manager (``with open(...)``) ensures a file is closed even if reading or writing fails. Text must be decoded consistently, so UTF-8 is specified. Exceptions separate expected data problems from program defects: an invalid number can become a documented missing value, but an absent required argument should stop execution with a useful message. Careful handling prevents one malformed value from silently corrupting the entire result.

CSV Parsing & Pandas

Manual CSV parsing exposes the structure hidden behind a convenience library. The first non-empty line supplies column names; each later line is split into fields and paired with that header to create a dictionary. Type conversion is a separate step because text read from a file does not automatically become an integer or Boolean category. The exercise is valuable for learning, although a simple comma split does not support every part of the CSV standard, such as quoted commas or embedded newlines.

Pandas loads the table into a DataFrame, where columns have inferred data types and rows can be filtered with expressions. Aggregations such as mean, minimum, maximum, and group-by summaries become concise operations. The manual and pandas implementations calculate total students, submission counts, average scores, extreme scorers, domain averages, missing submissions, and scores below five. A comparison report checks each result rather than assuming the two methods agree.

The comparison illustrates a broader engineering principle: convenience should not replace verification. Pandas reduces boilerplate and handles large, complex datasets effectively, while the manual version clarifies the underlying algorithm. Matching outputs provide a small form of cross-validation and increase confidence in both implementations.

Data Cleaning

Raw data often reflects several people, tools, and conventions. The supplied table contains an exact duplicate, inconsistent domains, percentage symbols, invalid attendance, number words, metre and centimetre heights, kilogram labels, mixed submission codes, and missing values. Cleaning turns these representations into a consistent schema before analysis.

The exact duplicate is removed first so it cannot distort averages or counts. Domain names are mapped to four controlled categories. Attendance is converted to a numeric percentage, and values below zero or above one hundred are treated as invalid. Scores and study hours convert recognized number words before numeric coercion. Heights are normalized to centimetres and weights to kilograms; names such as `'height_cm'` retain unit information in the schema. Submission variants such as Yes and Y become `'yes'`, while No and N become `'no'`.

Missing scores are filled with zero because the dataset treats a missing score as not scored. Missing attendance, study hours, and weight use the median. For a small dataset, the median is less sensitive than the mean to extreme values. This is a defensible project rule, but it is not universally correct; production work should preserve an imputation indicator and confirm the meaning with the data owner.

Validation checks that student IDs are unique, attendance lies in range, measurement columns are numeric, categories belong to allowed sets, and critical fields contain no nulls. The program refuses to publish the cleaned CSV if a check fails. The before-and-after JSON summaries and written report make the transformation traceable rather than presenting cleaned data as if it arrived in that form.

Workflow

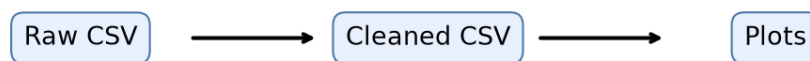


Figure 1: Data-processing workflow

Visualization

The domain-average bar chart compares submitted students after grouping by domain. It makes differences in performance easier to scan than a table of individual rows, while the zero-to-ten axis keeps the score scale honest. Electronics has the highest submitted-student average in this dataset; Mechanical has no submitted record and therefore does not appear in this particular aggregation.

The attendance-versus-score scatter plot retains one mark per student. It shows a positive association: higher attendance generally accompanies higher scores in this small sample. This observation is descriptive, not causal. Median imputation and the limited number of records also mean that a correlation should be interpreted cautiously.

The submission-status chart compares the number of 'yes' and 'no' records. Most students submitted their work, while a smaller group did not. This chart is operationally useful because it immediately communicates follow-up workload. Across all three plots, titles, labels, restrained colors, readable dimensions, and tight layouts support clear communication.

Saving the figures at a defined resolution also makes the analysis reproducible and suitable for this report.

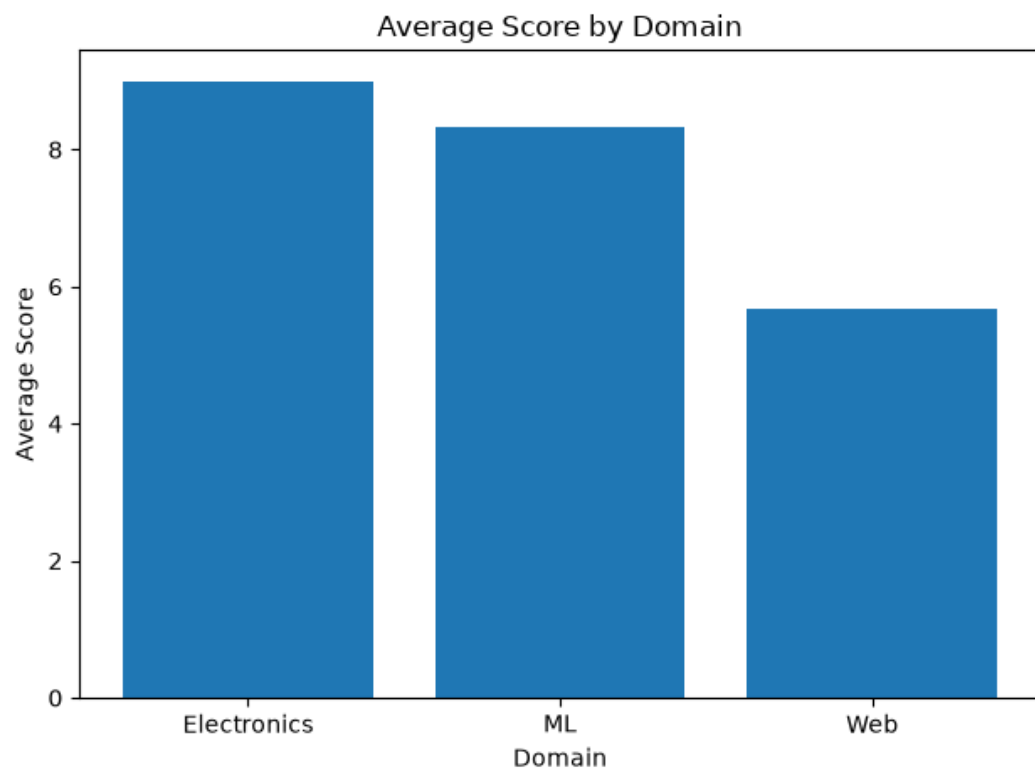


Figure 2: Average score by domain

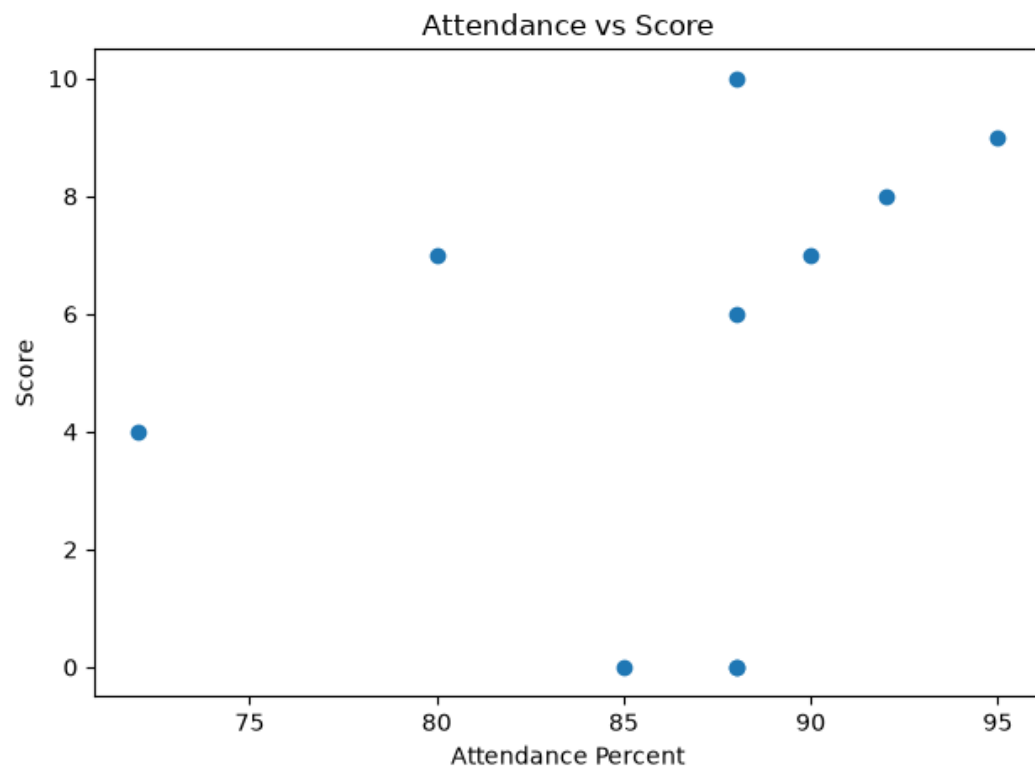


Figure 3: Attendance versus score

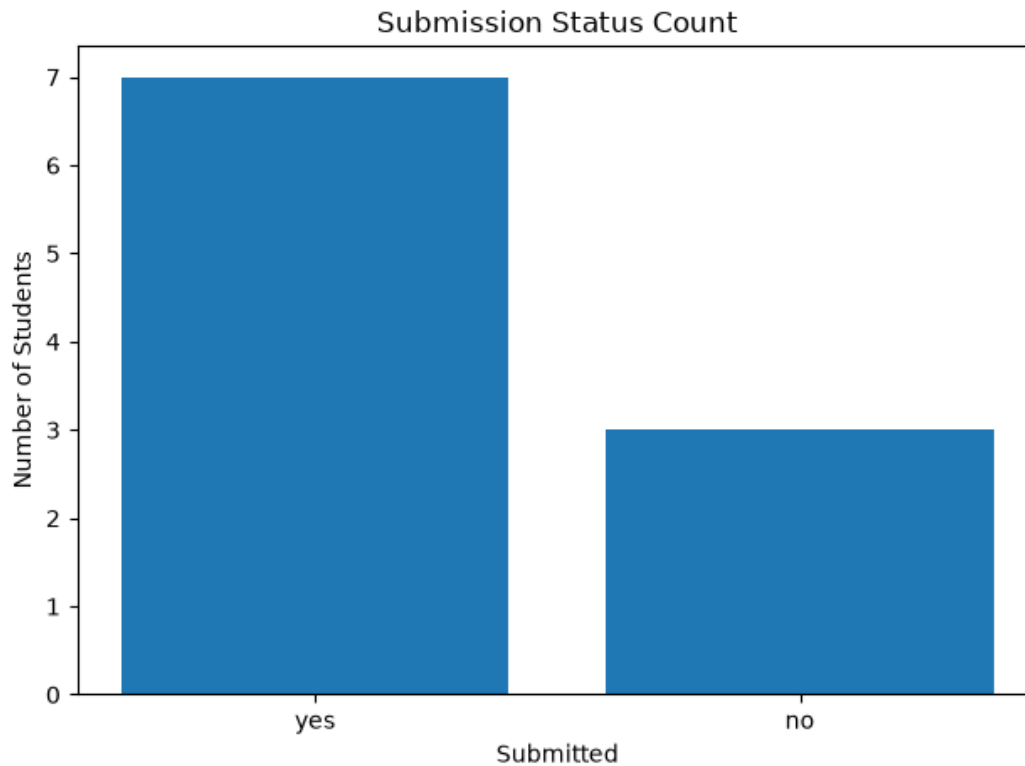


Figure 4: Submission-status count

Conclusion

Across Tasks 1–5, I learned that reliable analysis is a chain rather than a single script. Environment setup and dependency records make execution repeatable. Git captures intent and history. Core Python structures make data transformations explicit, while pandas provides expressive operations for tabular work. Manual parsing built understanding of what a library automates, and comparison exposed discrepancies instead of hiding them.

The cleaning task was the clearest reminder that data quality requires decisions. Units, categories, missing values, and invalid ranges must be handled according to documented rules, then tested. Visualization completed the workflow by translating the cleaned records into interpretable comparisons. The project can be extended with automated tests, a standards-compliant CSV parser for complex files, command-line argument parsing, richer logging, and larger datasets. Its current form nevertheless provides a complete, inspectable path from raw CSV to validated data and communication-ready plots.

References

Python Software Foundation. Python 3 Documentation. <https://docs.python.org/3/>

pandas development team. pandas Documentation. <https://pandas.pydata.org/docs/>

Matplotlib development team. Matplotlib Documentation. <https://matplotlib.org/stable/>

Git Project. Git Documentation. <https://git-scm.com/doc>